

Generation as Search Operator for Test-Time Scaling of Diffusion-Based Combinatorial Optimization (NeurIPS, 2025)

26.07.02

김정현

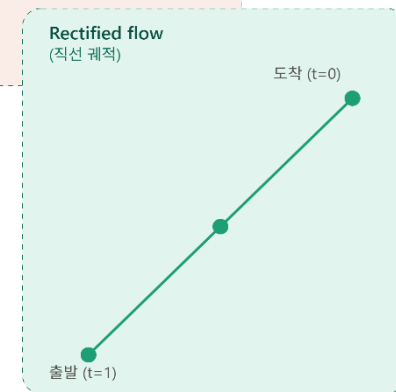
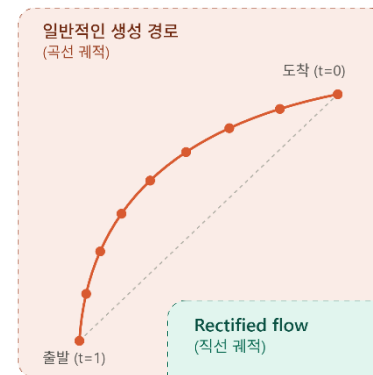


❖ Background: Combinatorial Optimization (CO)

- ✓ 수많은 가능한 선택지 중 제약 조건을 만족하는 '최적의 경로/순서'를 찾는 문제 (Ex: TSP, 공정 스케줄링)
- ✓ 문제 규모가 커질수록 가능한 경우의 수가 기하급수적으로 증가하여 현실적으로 풀기 어려움 (NP-hard)
- ✓ 최근 Neural Combinatorial Optimization이 등장 → 데이터 기반으로 최적화 정책을 학습
 - Constructive Methods (AR/NAR), Improvement-based Methods, Divide-and-Conquer, Generative Methods

❖ Diffusion Model

- ✓ 학습 시: 정답 데이터에 노이즈를 점점 추가해 완전히 무너뜨림 → 이 과정을 거꾸로 되돌리는 법(Denoising)을 학습
- ✓ 추론 시: 완전한 노이즈에서 출발 → 학습된 Denoising을 반복 적용 → 점점 그럴듯한 솔루션으로 수렴
- ✓ 최적해의 분포를 학습하여, 노이즈 상태(초기 값)로부터 점진적으로 최적의 솔루션 경로를 찾아 나가는 과정으로 활용
- ✓ Existing Methods Limitation:
 - Denoising 단계 수(T)를 늘려서 성능을 높이려 하지만 → 어느 순간부터 성능이 빠르게 정체(diminishing returns)
 - 매번 "완전한 노이즈"에서 새로 시작하기 때문에, 이전에 만든 괜찮은 해를 이어받아 개선하는 것이 구조적으로 어려움
 - 해를 개선하려고 별도 탐색(Gradient Search)을 추가하면 연산 비용이 커져 대규모 문제에는 비효율적
- ✓ Rectified Flow: 두 분포(예: 노이즈 ↔ 데이터, 혹은 나쁜 해 ↔ 좋은 해) 사이를 잇는 경로를 직선 궤적으로 학습
- ✓ → 궤적이 곧기 때문에 적은 스텝으로도 오차가 적은 고품질 결과 생성 가능
- ✓ → GenSCO는 "노이즈 → 해"가 아니라 "저품질 해" → "고품질 해"로 직접 이어주는 Solution-to-Solution 생성형 모델 설계



❖ Graph-based CO Formulation

- ✓ 그래프 $G(V, E)$ 기반의 최적화 문제 $\rightarrow$ 제약조건을 만족하는 해 중 목적함수를 최소화: $\min_{x \in \Omega} l(x; G)$
- ✓ Edge-decision: 간선(edge) 선택 여부를 결정 (Ex: TSP $\rightarrow$ 어떤 두 도시를 연결할지)
- ✓ Node-decision: 노드(node) 선택 여부를 결정 (Ex: MIS $\rightarrow$ 어떤 노드를 집합에 포함할지)

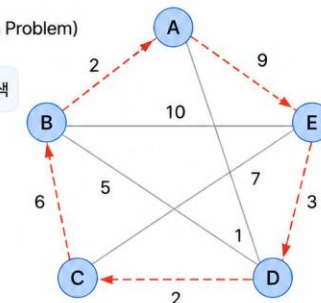
❖ Diffusion Modeling for CO

- ✓ 핵심 목표: 주어진 문제 인스턴스 G 가 주어졌을 때, 고품질 솔루션들의 분포 $p_\theta(x_o|G)$ 를 학습하는 것
- ✓ 학습 과정:
 - Forward: 정답 솔루션 x_0 에 노이즈를 단계적으로 추가해서 잠재변수 $x_{1:T}$ 를 생성
 - Reverse: 노이즈가 낀 상태에서 원래의 솔루션을 복원하는 과정을 학습
 - 모델이 생성한 분포 $p_\theta(x_o|G)$ 와 실제 데이터 분포 $q(x_o|G)$ 사이의 차이를 최소화하는 방향으로 학습
- ✓ Discrete Diffusion:
 - 솔루션을 원-핫 벡터로 표현하고 Multinomial Noise를 사용하여 확률적으로 변수를 무작위화
 - 전이 행렬을 통해 시간이 지날수록 균등 분포(uniform distribution)로 수렴
- ✓ Continuous Diffusion:
 - 이산적인 값을 연속 구간 $([-1, 1]^N)$ 으로 매핑하고 Gaussian Noise를 추가
 - 정규분포를 따라 점진적으로 변형하며, 최종적으로는 가우시안 분포로 수렴

1 TSP (Travelling Salesman Problem)

모든 도시를 한 번씩 방문하는 최단 경로 탐색

- 도시(노드)
- 거리(간선)
- -> 경로(예시)

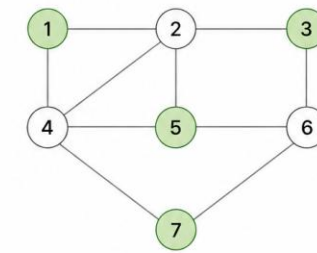


2 MIS (Maximum Independent Set)

(Maximum Independent Set)

서로 연결되지 않은 노드를 최대한 많이 선택

- 노드
- 간선
- 선택된 노드(예시)

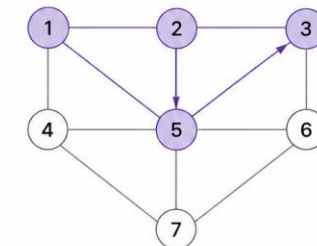


3 MCI (Maximum Clique)

(Maximum Clique)

모든 노드가 서로 연결된 가장 큰 그룹 선택

- 노드
- 간선
- 선택된 노드(예시)



● 도시(노드) ● MIS 선택 예시 ● MCI 선택 예시 — 간선 - -> 경로(예시)

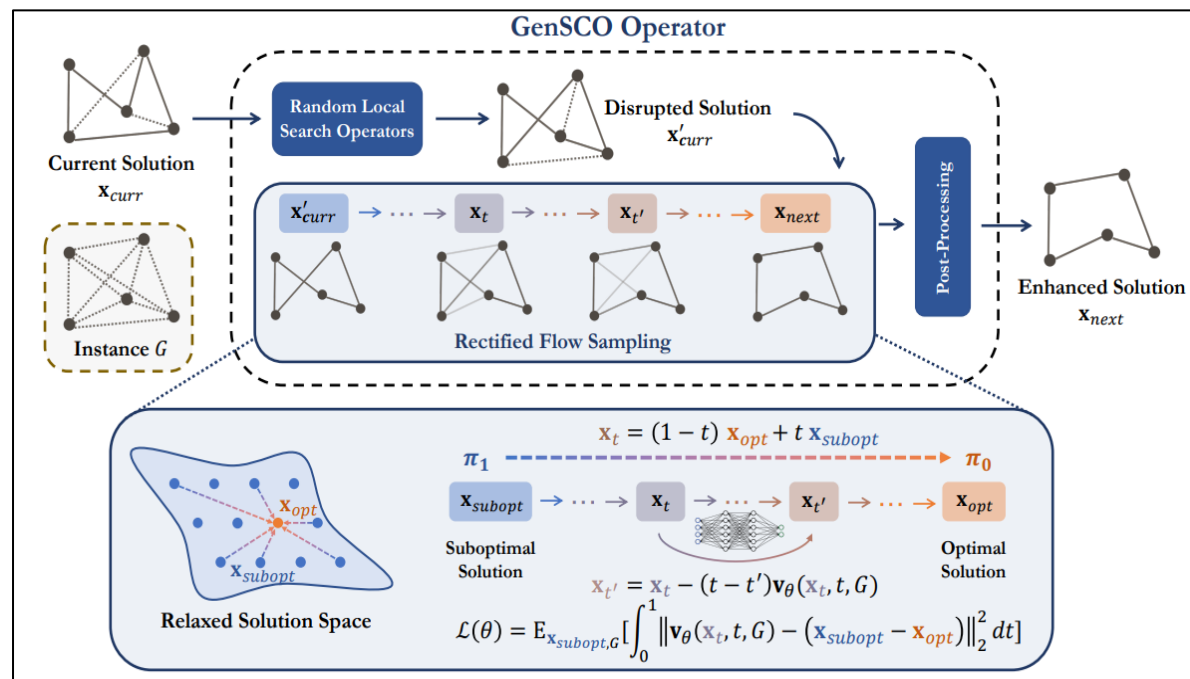
III The GenSCO Framework

❖ GenSCO: "Generation as a Search Operator"

- ✓ Denoising step만 늘리는 기존 방식은 성능이 빠르게 정체되고, 매번 노이즈부터 새로 생성하므로 기존 해를 이어받아 개선하기 어려움
- ✓ 핵심 아이디어: Diffusion의 생성 과정을 "한 번에 완성하는 생성기"가 아니라, 반복 적용 가능한 탐색 연산자(search operator)로 재정의
- ✓ 전략: 매 회 완벽한 해를 만들려 하기보다, 가볍고 저렴한 개선을 여러 번 반복(Refinement Cycles) 해서 누적된 성능 향상을 얻음

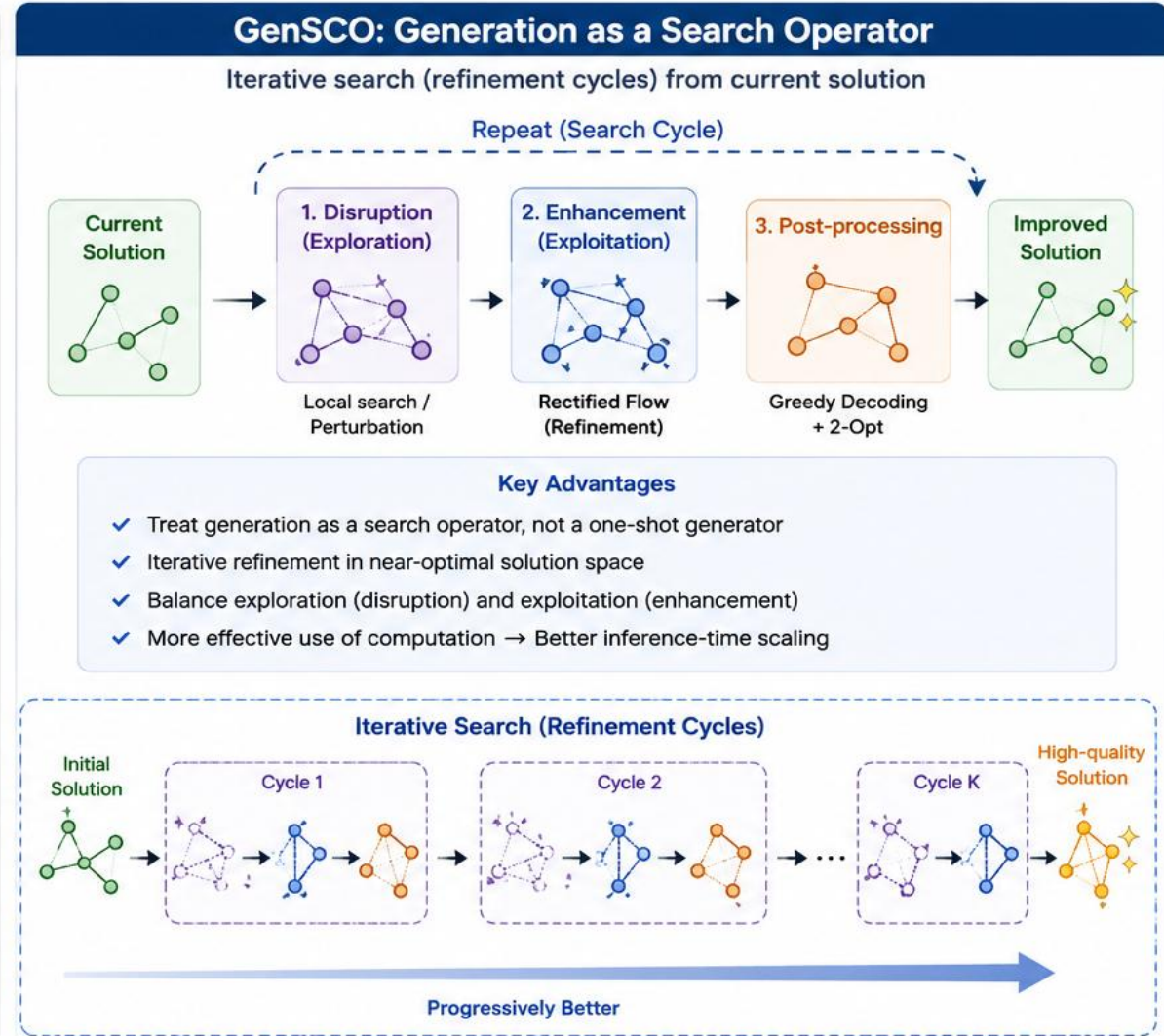
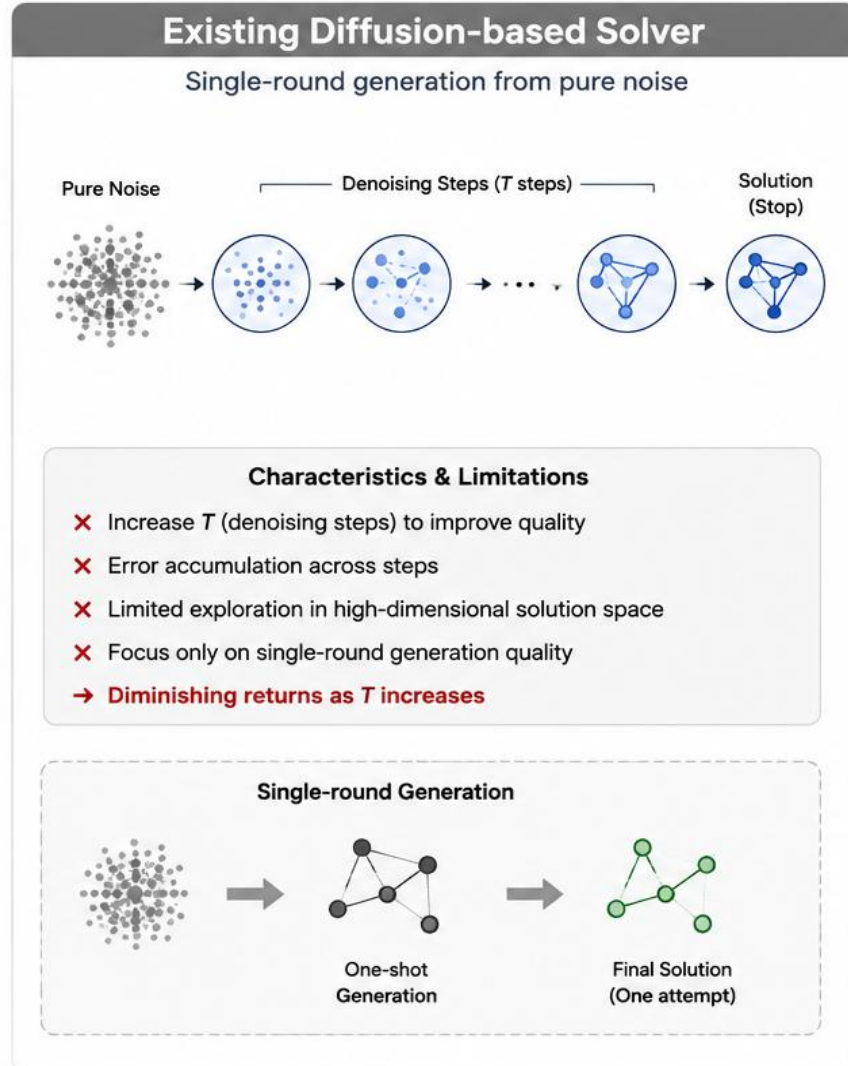
❖ GenSCO Search Cycle

- ✓ 한 사이클은 다음 세 단계로 구성되며, 이 사이클을 N_c 번 반복
 - ① Disruption(교란): Local search operator(Random 2-Opt)로 현재 해를 의도적으로 흐트러뜨려 local optimum에서 탈출
 - ② Enhancement(개선): Rectified Flow sampling으로 교란된 해를 다시 좋은 해 방향으로 개선
 - ③ Post-processing: Greedy decoding + 2-Opt로 실행 가능한 (feasible) solution으로 변환
- ✓ 매 사이클 종료 시 solution의 objective 값을 평가하고, 지금까지 찾은 것 중 최고 solution을 계속 추적
 - ✓ 전체 탐색 과정은 병렬 실행도 가능 → 여러 연산자를 동시에 굴러 탐색 효율 극대화



III The GenSCO Framework

❖ Comparison of Existing Diffusion Solver and GenSCO



❖ Solution-to-Solution Learning

- ✓ 기존 diffusion은 "노이즈 → 해"를 학습하지만, GenSCO는 "suboptimal solution → optimal solution" 을 직접 학습
 - Local search로 suboptimal solution 생성
 - TSP: Random 2-Opt / MIS-MCI: Variable Flipping → $N/4 \sim 3N/4$
- ✓ 두 solution 사이를 직선 경로(linear trajectory) 로 정의:
 - $x_t = tx_{subopt} + (1 - t)x_{opt}, t \in [0,1]$
 - $t = 0$ 일 때 $x_{opt}, t = 1$ 일 때 x_{subopt}
 - 경로가 직선이므로 적은 sampling step으로도 효율적인 refinement 가능

❖ Rectified Flow Optimization

- ✓ 이 직선 경로를 따라 이동하는 방향(velocity)은 시간 t 에 관계없이 일정
 - $v(x_t, t, G) = x_{subopt} - x_{opt}$
- ✓ 신경망 v_θ 가 이 실제 방향을 예측하도록 학습:
 - $L(\theta) = E_{x_{subopt}, G} \left[\int_0^1 \|v_\theta(x_t, t, G) - v(x_t, t, G)\|_2^2 dt \right]$
 - 단, 추론 시에는 x_{opt} (정답)을 알 수 없으므로, 신경망이 x_{opt} 자체를 예측하도록 재구성: $v_\theta(x_t, t, G) = x_{subopt} - f_\theta(x_t, t, G)$
- ✓ Euler Method를 이용하여 Solution 반복 업데이트
 - $x_{\tau_{n+1}} = x_{\tau_n} - (\tau_n - \tau_{n+1})v_\theta(x_{\tau_n}, \tau_n, G)$

Algorithm 1 GenSCO Framework for Solving

Input: Flow model $v_\theta(\cdot, \cdot, \cdot)$, graph problem instance G , number of cycles N_c , sequence of time points $1 = \tau_1 > \tau_2 > \dots > \tau_{N_\tau} = 0$

Randomly initialize solution $\mathbf{x}$

for cycle $c = 1$ to N_c **do**

$\mathbf{x}_{\tau_1} \leftarrow \text{Disrupt}(\mathbf{x})$

for time step $n = 1$ to N_τ **do**

$\mathbf{d}_{\tau_n} = v_\theta(\mathbf{x}_{\tau_n}, \tau_n, G)$

$\mathbf{x}_{\tau_{n+1}} = \mathbf{x}_{\tau_n} - (\tau_n - \tau_{n+1})\mathbf{d}_{\tau_n}$

end for

$\mathbf{x} \leftarrow \text{PostProcess}(\mathbf{x}_{\tau_{N_\tau}})$

end for

Output: Solution $\mathbf{x}$

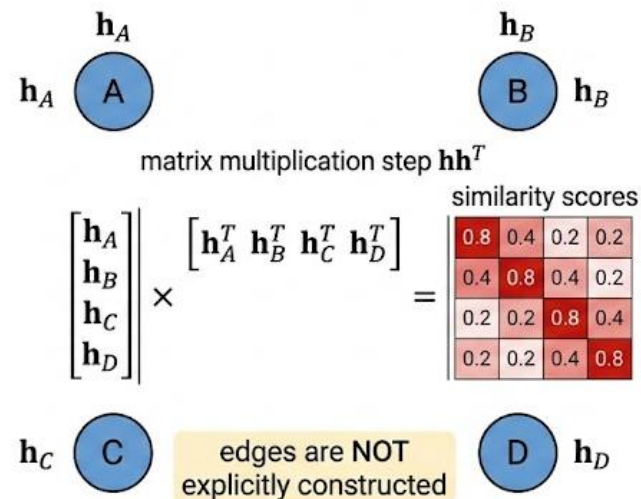
❖ Lightweight Architecture

- ✓ 왜 가벼운 구조가 필요한가?
 - $f_{\theta}(x_t, t, G)$ - "현재 상태를 보고 최적해를 예측하는 신경망"
 - 이 신경망을 탐색 전체에서 수백~수천 번 반복 호출해야 하므로, 모델 하나가 무거우면 전체 탐색 속도가 크게 느려짐
- ✓ 기존 Diffusion Architecture의 한계:
 - Edge Feature 기반 GNN 사용 $\rightarrow$ 모든 노드 쌍(edge)에 대한 정보를 명시적으로 계산 $\rightarrow O(N^2)$ 의 높은 계산 복잡도
 - 정확도는 높지만, 반복적인 Denoising·Search Cycle에서는 계산 비용이 누적되어 비효율적
- ✓ Proposed Architecture (GenSCO):
 - Node feature만 사용하는 Transformer 기반 구조 $\rightarrow$ edge feature를 아예 계산하지 않음
 - 대신 Self-Attention 계산에 adjacency matrix A 를 직접 더해 그래프 연결 정보를 주입
 - $\rightarrow \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \lambda A\right)V$
 - TSP에서는 두 노드 임베딩의 inner product (hh^T)로 두 노드가 연결될 확률(heatmap) 계산

❖ Advantages

- ✓ Edge feature를 제거하여 연산 부담 감소
- ✓ Lightweight 구조로 수백~수천 번의 Solution Refinement 수행 가능
- ✓ Adjacency Matrix로 그래프 연결 관계 반영

GenSCO Transformer with inner product (hh^T)



❖ Dataset & Evaluation

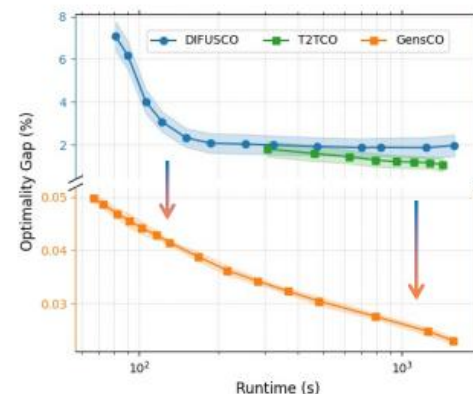
- ✓ 데이터: 단위 정사각형 $[0, 1]^2$ 내에서 N 개의 노드 샘플링
- ✓ 학습 데이터: 1,280K (TSP-100), 128K (TSP-500/1000)
- ✓ 테스트 데이터: 1,280 (TSP-100), 128 (TSP-500/1000)
- ✓ Ground Truth: Concorde (TSP) 및 KaMIS (MIS/MCI) 활용
- ✓ 평가 지표: 목적 함수 값(Distance/Cost), 최적성 갭(Gap %), 실행 시간(Time)
- ✓ 총 15개 이상의 베이스라인과 비교 (Exact, Heuristic, Non-generative, Generative 모델 포함)

❖ GenSCO 모델 설정

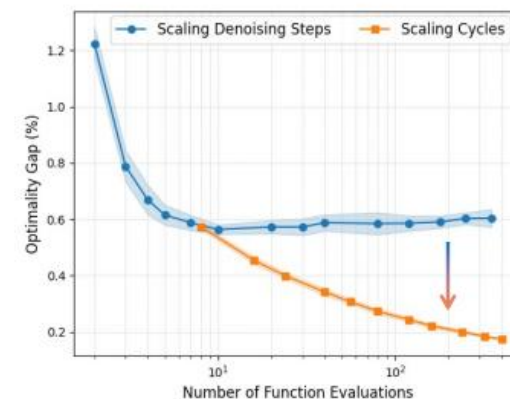
- ✓ Denoising step ($N_\tau = 4$): Euler method 반복 횟수
- ✓ 병렬 연산자 8개 실행: 같은 문제를 8개 경로로 동시 탐색해 GPU 활용도 극대화
- ✓ 탐색 주기 (C): Search Cycle 반복 횟수 / 2-Opt 후처리: $N/50$ 단계

❖ Computational Resources

- ✓ 학습 환경: NVIDIA RTX 4090 (24GB) 사용
- ✓ GenSCO: TSP-100(≤ 2 일), TSP-500/1000(≤ 1 일)
- DIFUSCO 대비 훈련 시간 대폭 단축 (DIFUSCO는 A100 사용 시 최대 8.6일 소요)



(a) Scaling w.r.t. Runtime



(b) Scaling w.r.t. NFEs

❖ Main Results

- ✓ LKH3 대비 TSP-100에서 141배 빠른 0.000% Gap 달성
- ✓ DIFUSCO 대비 70배 이상의 갭 감소 및 60배 수준의 속도 향상
- ✓ 2-Opt 추가 적용 시 시간은 비슷하되 Gap 추가 개선
- ✓ C(탐색 주기)가 커질수록 성능은 좋아지지만 시간도 비례해 증가

❖ Cross-Scale Generalization

- ✓ 동일 스케일 또는 더 큰 스케일로 테스트 시 기존 대비 확실한 우위
- ✓ 분포 차이가 큰 경우 성능 저하가 일부 발생하나, 기존 방법들보다 일관되게 높은 성능을 보임

Testing \ Training		TSP-100	TSP-500	TSP-1000
TSP50	DIFUSCO ($T_s=50$) [21]	5.70, 0.25%	5.83, 2.55%	5.84, 2.71%
	T2T ($T_s=50, T_g=30$) [9]	5.70, 0.11%	5.78, 1.60%	5.75, 1.10%
	GenSCO (C=40)	5.69, 0.00%	5.69, 0.05%	5.69, 0.08%
TSP100	DIFUSCO ($T_s=50$) [21]	7.78, 0.23%	8.03, 3.44%	8.02, 3.31%
	T2T ($T_s=50, T_g=30$) [9]	7.77, 0.08%	7.95, 2.47%	7.91, 1.96%
	GenSCO (C=40)	7.76, 0.00%	7.85, 1.16%	7.87, 1.42%
TSP500	DIFUSCO ($T_s=50$) [21]	17.05, 3.04%	16.78, 1.40%	16.86, 1.85%
	T2T ($T_s=50, T_g=30$) [9]	16.92, 2.25%	16.68, 0.81%	16.72, 1.00%
	GenSCO (C=40)	16.99, 2.70%	16.55, 0.01%	16.55, 0.02%
TSP1K	DIFUSCO ($T_s=50$) [21]	24.04, 3.98%	23.65, 2.30%	23.63, 2.21%
	T2T ($T_s=50, T_g=30$) [9]	23.85, 3.16%	23.47, 1.51%	23.41, 1.23%
	GenSCO (C=40)	24.37, 5.40%	23.14, 0.11%	23.13, 0.05%

Table 1: Results on TSP. AS: Active Search, G: Greedy, S: Sampling Decoding, BS: Beam Search.

Method	TSP-100 (1280 inst.)			TSP-500 (128 inst.)			TSP-1000 (128 inst.)		
	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time
<i>Mathematical Solvers or Heuristics</i>									
Concorde [79]	7.76	0.000%	5.1m	16.55	0.000%	39.8m	23.12	0.000%	3.0h
LKH-3 [80] (trials=32)	7.76	0.009%	52s	16.55	0.024%	42s	23.13	0.035%	2.2m
LKH-3 [80] (trials=64)	7.76	0.006%	59s	16.55	0.015%	50s	23.12	0.025%	2.5m
LKH-3 [80] (trials=128)	7.76	0.003%	1.2m	16.55	0.011%	1.1m	23.12	0.015%	3.1m
LKH-3 [80] (trials=256)	7.76	0.002%	1.6m	16.55	0.006%	1.6m	23.12	0.009%	4.1m
LKH-3 [80] (trials=512)	7.76	0.001%	2.5m	16.55	0.002%	2.5m	23.12	0.005%	6.2m
LKH-3 [80] (trials=4096)	7.76	0.000%	14.1m	16.55	0.000%	15.0m	23.12	0.001%	36.3m
<i>Non-Generative Neural Solvers</i>									
AM [1] BS	7.95	2.48%	12.9m	19.53	18.03%	2.8m	29.90	29.24%	12.6m
POMO [2] $\times 8$ augment	7.77	0.13%	8s	20.22	22.19%	1.0m	32.55	40.57%	8.0m
POMO+EAS [81] AS+G	7.76	0.05%	38.4m	24.54	48.22%	11.6h	49.56	114.36%	63.5h
GCN [8] BS	8.41	8.38%	6.0m	30.37	83.55%	38.0m	51.26	121.73%	51.7m
DIMES [12] S+2Opt	7.98	2.88%	5.0m	17.64	6.56%	1.1m	24.81	7.29%	2.9m
DIMES [12] AS+S+2Opt	7.93	2.18%	3.2h	17.29	4.48%	2.1h	24.32	5.17%	4.5h
LEHD [82] PRC 100	7.76	0.01%	1.8m	16.61	0.34%	8.0m	23.44	1.22%	43.0m
LEHD [82] PRC 1000	7.76	0.002%	16.9m	16.58	0.17%	1.2h	23.32	0.72%	7.0h
BQ-NC0 [22] BS	7.76	0.01%	4.1m	16.64	0.55%	15m	23.47	1.38%	38m
GLOP [39]	—	—	—	16.91	1.99%	1.5m	23.84	3.11%	3.0m
UDC [40]	—	—	—	16.78	1.58%	4.0m	23.53	1.78%	8.0m
COExpander [53]	7.76	0.01%	3.84m	16.59	0.25%	1.41m	23.27	0.64%	5.2m
<i>Generative Neural Solvers</i>									
DIFUSCO ($T_s=50$) [21] G+2Opt	7.78	0.28%	18.9m	16.80	1.50%	4.7m	23.55	1.89%	14.4m
DIFUSCO ($T_s=100$) [21] S+2Opt	7.76	0.06%	30m	16.69	0.87%	19.1m	23.42	1.31%	51.9m
T2T ($T_s=50, T_g=30$) [9] G+2Opt	7.76	0.07%	28.6m	16.68	0.82%	6.5m	23.44	1.40%	19.7m
T2T ($T_s=50, T_g=30$) [9] S+2Opt	7.76	0.02%	45.1m	16.63	0.48%	19.7m	23.37	1.07%	51.1m
Fast T2T ($T_s=5, T_g=5$) [24] G+2Opt	7.76	0.03%	4.2m	16.61	0.39%	2.2m	23.25	0.58%	8.6m
Fast T2T ($T_s=5, T_g=5$) [24] S+2Opt	7.76	0.01%	8.3m	16.58	0.21%	6.9m	23.22	0.42%	18.3m
GenSCO (C=10)	7.76	0.000%	6s	16.55	0.023%	5s	23.25	0.559%	12s
GenSCO (C=20)	—	—	—	16.55	0.020%	9s	23.21	0.390%	23s
GenSCO (C=40)	—	—	—	16.55	0.016%	18s	23.18	0.258%	46s
GenSCO (C=10) 2Opt	7.76	0.000%	6s	16.55	0.019%	5s	23.13	0.063%	16s
GenSCO (C=20) 2Opt	—	—	—	16.55	0.016%	9s	23.13	0.054%	30s
GenSCO (C=40) 2Opt	—	—	—	16.55	0.012%	18s	23.13	0.046%	58s
GenSCO (C=80) 2Opt	—	—	—	16.55	0.011%	36s	23.13	0.041%	2.0m
GenSCO (C=160) 2Opt	—	—	—	16.55	0.010%	1.2m	23.13	0.036%	3.9m

IV Experiments

❖ Self-improving Capability

- ✓ 완벽한 최적해(Optimal)가 아닌, 고전 휴리스틱(2-Opt)으로 생성된 저품질 데이터로 학습 가능
- ✓ 학습된 모델로 데이터셋의 일부(1/75)를 고품질 데이터로 재라벨링
- ✓ 개선된 데이터를 이용해 모델을 순차적으로 미세 조정 (GenSCO → GenSCO2 → GenSCO3)

Method	TSP-100		TSP-500	
	Obj.	Gap	Obj.	Gap
2Opt	7.905	1.920%	17.703	6.994%
GenSCO	7.757	0.014%	16.823	1.676%
GenSCO ²	7.756	0.007%	16.699	0.927%
GenSCO ³	7.756	0.005%	16.642	0.583%

❖ Ablation Study

- ✓ Parallel Execution:
 - 병렬 연산자 수(Runs)가 늘수록 탐색 다양성 확보 → Gap 감소 but 연산시간도 증가
- ✓ Architecture: Transformer vs. GCN:
 - TSP-1000, C=160 기준 GCN 27분 20초 vs Transformer 58초
 - → 약 28배 속도 차이 (압도적 우위)
- ✓ Flow vs Consistency Model:
 - Consistency+2Opt 대비 약 4~5배 빠르면서 비슷하거나 더 나은 정확도 달성
- ✓ Disruption Operator:
 - 교란 없이는 C를 늘려도 성능이 정체
 - (TSP-500: 0.083%→0.056%)
 - 교란 있으면 계속 개선 (0.063%→0.046%)

Method	Runs	TSP-500			TSP-1000		
		Obj.	Gap (%)	Time (s)	Obj.	Gap (%)	Time (s)
GenSCO (C=40)	1	16.550	0.028	2.408	23.137	0.083	7.609
	2	16.549	0.021	4.383	23.133	0.065	12.372
	4	16.548	0.016	9.248	23.130	0.053	24.164
	8	16.548	0.013	17.974	23.129	0.047	47.468
GenSCO (C=80)	1	16.549	0.021	4.365	23.134	0.067	14.012
	2	16.549	0.016	8.333	23.131	0.055	23.834
	4	16.548	0.012	17.904	23.129	0.047	47.327
	8	16.548	0.011	35.541	23.127	0.040	93.989
GenSCO (C=160)	1	16.548	0.016	8.226	23.131	0.056	27.741
	2	16.548	0.013	16.160	23.129	0.047	46.770
	4	16.548	0.011	35.375	23.128	0.041	93.904
	8	16.548	0.010	70.923	23.126	0.034	187.634

Method	Obj.	TSP-500		Time	Obj.	TSP-1000		Time
		Gap	Time			Gap	Time	
GenSCO (C=40) with GCN	16.556	0.060%	1m45s		23.157	0.167%	6m50s	
GenSCO (C=40) with Transformer	16.549	0.019%	5s		23.133	0.063%	16s	
GenSCO (C=80) with GCN	16.553	0.045%	3m25s		23.147	0.127%	13m40s	
GenSCO (C=80) with Transformer	16.548	0.016%	9s		23.131	0.054%	30s	
GenSCO (C=160) with GCN	16.552	0.037%	6m50s		23.140	0.097%	27m20s	
GenSCO (C=160) with Transformer	16.548	0.012%	18s		23.129	0.046%	58s	

Method	Obj.	NFE=40		Time	Obj.	NFE=80		Time	Obj.	NFE=160		Time
		Gap	Time			Gap	Time			Gap	Time	
Consistency [24]	16.598	0.313%	7s		16.590	0.265%	14s		16.584	0.233%	28s	
Consistency [24] 2Opt	16.550	0.028%	23s		16.549	0.022%	46s		16.549	0.021%	1m33s	
Flow	16.550	0.023%	5s		16.549	0.020%	9s		16.548	0.016%	18s	
Flow 2Opt	16.549	0.019%	5s		16.548	0.016%	9s		16.548	0.012%	18s	

❖ Key Contributions

- ✓ 패러다임 전환: 단일 생성 품질 극대화 → 추론 시간의 비용 효율적 탐색(Search-driven Scaling)
- ✓ 시스템 재설계: solution-to-Solution Rectified Flow + 조절 가능한 탐색 연산자
- ✓ 성능 개선: 기존 최신 뉴럴 솔버 대비 압도적 성능 달성 및 LKH3를 능가하는 속도와 정확도 확보

❖ Technical Highlights

- ✓ Generation as Search: 생성을 1회성 출력이 아닌 반복적 탐색/개선 과정으로 재정의
- ✓ Bias-Rectified Training: 학습 단계에서 미리 "저품질→고품질" 변환 경로를 학습
- ✓ Scalable Architecture: 경량 트랜스포머 아키텍처를 도입하여 대규모 인스턴스에서도 안정적이고 빠른 확장성 유지

❖ Limitations & Future Work

- ✓ 한계점: 대규모 문제로 갈수록 성능 향상 속도가 점진적으로 둔화되어 LKH 대비 경쟁력이 약화되는 구간이 존재
- ✓ 일부 복잡한 실제 데이터(TSPLIB)에서 일반화 성능 저하 및 강건성 부족 문제가 관찰됨
- ✓ 향후 연구:
 - ✓ 대규모 최적화: 초대규모 데이터에서도 고효율 탐색을 지속할 수 있는 확장형 알고리즘 설계
 - ✓ 하이브리드 전략: 뉴럴 솔버의 속도와 전통적 솔버의 정밀도를 결합한 2단계(Two-stage) 해결 방식 탐구
 - ✓ 강건성 강화: 데이터 분포 변화에 유연하게 대응하는 차세대 뉴럴 아키텍처 고도화